GPUMODE Lecture Study Notes: Lecture 37
SASS and GPU Microarchitecture

Core Problem the Lecture Addresses

This lecture addresses the gap between writing CUDA, Triton, PyTorch, or compiler-generated GPU code and understanding the actual machine instructions that execute on NVIDIA GPUs. The central idea is that most engineers should not try to hand-write SASS directly. Instead, they should understand SASS and GPU microarchitecture well enough to diagnose compiler output, reason about bottlenecks, and write higher-level code that guides the compiler toward better machine code.

SASS is NVIDIA's real GPU machine assembly. It is lower level than PTX. PTX is a virtual instruction set that is usually consumed by `ptxas`, NVIDIA's optimizing backend compiler. The GPU does not execute PTX directly in normal optimized execution. The GPU executes SASS.

The main technical problem is:

> High-level GPU code only reaches peak performance when the programmer understands what the compiler and hardware are likely to do below the source level. SASS inspection is therefore not mainly about hand-writing assembly; it is about closing the feedback loop between source code, compiler behavior, profiler counters, and hardware execution.

The lecture motivates this with several practical observations:

- Triton and CUDA can often outperform naive hand-written low-level code because they provide better iteration time and strong compiler optimizations.
- Extreme hand optimization can eventually beat higher-level code, but the cost is large.
- For large-scale training or inference, even a small performance-per-watt improvement can be economically meaningful.

- PTX is not enough for performance reasoning because `ptxas` may transform PTX into substantially different SASS.
- Nsight Compute, Compiler Explorer, `cuobjdump`, and `nvdisasm` all expose different parts of the source-to-hardware story.

Required Background

CUDA Compilation Pipeline

CUDA code typically passes through several conceptual representations:

1. CUDA C++ or generated CUDA-like code.
2. PTX, which is a virtual GPU instruction set.
3. SASS, which is the real target-specific machine code.
4. Runtime execution on SMs, warps, register files, caches, memory pipelines, tensor core pipelines, and special function units.

A simplified compilation model is:

$$\text{CUDA C++} \rightarrow \text{PTX} \rightarrow \text{SASS} \rightarrow \text{GPU execution.}$$

For Triton, the path is different at the front end, but it still ultimately reaches PTX or lower-level compiler representations before generating SASS:

$$\text{Triton Python} \rightarrow \text{Triton IR} \rightarrow \text{LLVM or MLIR path} \rightarrow \text{PTX} \rightarrow \text{SASS.}$$

The key point is that PTX is not a faithful one-to-one description of the instructions that execute. It is closer to a compiler intermediate representation.

PTX Versus SASS

PTX is forward compatible. A PTX program generated for an older architecture can often run on a newer GPU because the driver or `ptxas` can JIT-compile it to the newer target.

SASS is architecture-specific. SASS generated for one architecture may not be valid or optimal for another architecture.

This matters because a PTX instruction may map to:

- one SASS instruction,
- several SASS instructions,
- a completely different instruction sequence,

- a slow emulation path if the target hardware lacks native support,
- or a fused instruction if the backend compiler recognizes a pattern.

A useful mental model is:

PTX expresses intent, while SASS exposes execution.

GPU Execution Terminology

- **Thread**: the CUDA programming abstraction for one logical lane of work.
- **Warp**: a group of 32 CUDA threads executing under SIMT control.
- **Thread block**: a group of threads scheduled onto one SM.
- **SM**: streaming multiprocessor, the main execution unit of an NVIDIA GPU.
- **SM partition**: a subpartition inside an SM with its own warp scheduler and execution pipelines.
- **Register**: a 32-bit storage slot local to a thread.
- **Uniform register**: a register holding a value common to a warp, introduced in newer NVIDIA architectures.
- **Global memory**: HBM or device DRAM.
- **Shared memory**: on-chip programmer-managed memory inside an SM.
- **L1 and L2 cache**: hardware caches between SMs and HBM.
- **Occupancy**: the amount of active warps or thread blocks resident on an SM.
- **Instruction-level parallelism**: the amount of independent work within a warp that can overlap latency.
- **Thread-level parallelism**: the amount of independent warps or thread blocks available to hide latency.

Main Concepts

Why Study SASS If You Usually Cannot Write It Directly?

The lecture emphasizes that on modern NVIDIA GPUs, especially H100-class hardware, fully open-source SASS assembly support is incomplete. Tools like `QAssembler` exist but may not support all required instructions or encodings.

Therefore, the purpose of studying SASS is not necessarily to hand-write all kernels in assembly. The purpose is to understand the generated machine code.

This is similar to knowing CPU assembly when writing C++:

- You rarely write all code in assembly.
- You inspect assembly when performance matters.
- You adjust source code to make compiler output better.
- You use assembly knowledge to interpret profiler output.

For GPU kernels, this is even more valuable because small changes in source code can alter register pressure, occupancy, memory coalescing, instruction scheduling, branch structure, and pipeline utilization.

Iteration Time as the Real Optimization Metric

The lecture frames performance optimization around **iteration time**. For a training or inference system, iteration time includes:

- time to write code,
- time to compile,
- time to profile,
- time to debug correctness,
- time to run experiments,
- and time to complete the actual workload.

A useful model is:

$$T_{\text{iteration}} = T_{\text{implementation}} + T_{\text{compile}} + T_{\text{debug}} + T_{\text{profile}} + T_{\text{run}}.$$

Low-level SASS or CUDA optimization may reduce T_{run}, but it can drastically increase $T_{\text{implementation}}$, T_{debug}, and T_{profile}. For short experiments, PyTorch or Triton may win because they provide much faster iteration. For huge production workloads, low-level optimization becomes more attractive.

Optimization Per Nuclear Reactor

The lecture introduces a memorable metric: **optimization per nuclear reactor**. The idea is that at hyperscale, a 1% performance-per-watt improvement can correspond to enormous power savings.

Let:

$$P_{\text{cluster}}$$

be the total cluster power, and let r be the fractional efficiency improvement. Then the saved power is:

$$P_{\text{saved}} = rP_{\text{cluster}}.$$

For $r = 0.01$:

$$P_{\text{saved}} = 0.01P_{\text{cluster}}.$$

If the fleet consumes 10 GW, then:

$$P_{\text{saved}} = 0.01 \times 10\,\text{GW} = 100\,\text{MW}.$$

If a small modular nuclear reactor is around 50 MW, then:

$$100\,\text{MW} \approx 2 \text{ small modular reactors.}$$

This is not meant as a precise engineering cost model. It is a way to internalize why tiny kernel improvements can matter at very large scale.

PTX Forward Compatibility Can Hide Slow Paths

PTX instructions are designed to remain supported over time. This is convenient, but it can hide performance traps.

For example, if an older PTX instruction expresses an operation that the target hardware no longer supports natively, `ptxas` may emulate it using many SASS instructions. The program remains correct, but performance may collapse.

The lecture mentions low-bit matrix multiply as an example class. A PTX-level operation may exist, but on a newer GPU it may not map to native hardware and may be expanded into a long scalar sequence.

The lesson is:

> Correct PTX does not imply efficient SASS. Always inspect the generated SASS when a kernel unexpectedly underperforms.

Useful Tools for SASS Inspection

Compiler Explorer

Compiler Explorer, also known as Godbolt, provides a fast way to inspect PTX and SASS for small CUDA examples. Its strength is very fast iteration and source-to-assembly correlation.

Use it for:

- checking how small code changes affect SASS,
- comparing architectures such as Volta, Ampere, Ada, and Hopper,

- seeing source-line correlations,
- testing inline PTX,
- exploring compiler transformations.

Nsight Compute

Nsight Compute is the most important tool when profiling real kernels. It connects SASS with runtime measurements such as:

- warp stall reasons,
- instruction counts,
- memory throughput,
- cache behavior,
- source line correlation,
- register usage,
- achieved occupancy,
- issue rates,
- and pipeline utilization.

A typical optimization workflow is:

write kernel → profile in Nsight Compute → inspect SASS → change source → repeat.

`cuobjdump` and `nvdisasm`

NVIDIA provides official disassembly tools:

- `cuobjdump` extracts embedded device code from binaries.
- `nvdisasm` disassembles cubin files into SASS.

A representative workflow is:

```
nvcc -arch=sm_90 kernel.cu -o kernel
cuobjdump --dump-sass kernel > kernel.sass
cuobjdump --dump-ptx kernel > kernel.ptx
```

For more detailed disassembly, one may use `nvdisasm` on cubin files.

Open-Source SASS and Driver Projects

The lecture mentions several relevant projects:

- `maxas`, historically important for Maxwell-era SASS work.

- `QAssembler`, a newer open-source assembler with incomplete coverage for modern GPUs.
- `Nervana assembler` and similar older projects.
- `NVK`, an open-source NVIDIA Vulkan driver stack with a custom compiler backend.
- `nvdisasm` and reverse-engineering projects that document instruction encodings.

The important warning is that unofficial documentation may be subtly wrong. Microarchitectural details are difficult to infer perfectly from the outside.

SASS Instruction Concepts

Fixed-Length Instructions

Modern NVIDIA SASS instructions are typically fixed-width. The lecture highlights a 128-bit instruction encoding model for recent generations.

A fixed instruction length simplifies some hardware and compiler logic, but it reduces code density. Lower code density can matter when instruction cache behavior is important, especially for irregular control flow.

For machine learning kernels, control flow is often simple and straight-line, so the lower code density is usually not the dominant bottleneck. For ray tracing or branch-heavy workloads, it can matter more.

Register Size and Wide Values

NVIDIA registers are 32 bits. Larger values occupy multiple consecutive registers.

For a 64-bit pointer:

$$\text{pointer} = \text{low 32 bits} + 2^{32} \cdot \text{high 32 bits}.$$

A 64-bit address therefore needs two 32-bit registers. If a SASS instruction writes a 64-bit value, it often writes two consecutive registers such as `R2` and `R3`.

This matters for:

- address calculation,
- register pressure,
- liveness analysis,
- spilling,
- and dependency tracking.

Special Registers

CUDA thread and block indices originate from special hardware registers. A SASS instruction such as `S2R` reads a special register into a general register.

A simplified CUDA indexing expression is:

$$i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}.$$

In SASS, this often involves:

- reading thread index from a special register,
- reading block index from a special register,
- loading block dimension from constant or uniform state,
- multiplying block index by block dimension,
- adding thread index,
- multiplying by element size for byte addressing.

For a float array, the byte offset is:

$$\text{byte offset} = 4i.$$

The final address is:

$$\text{address} = \text{base pointer} + 4i.$$

Constant Memory and Kernel Arguments

Kernel parameters are often passed through constant memory. Older architectures could use constant-memory operands directly inside some arithmetic instructions. Newer architectures increasingly load constants into registers or uniform registers first.

A conceptual model is:

kernel argument $\rightarrow$ constant memory $\rightarrow$ register or uniform register $\rightarrow$ arithmetic or memory instruction.

This matters because constant loads may have latency. In rare cases, a constant-memory access can miss and become very expensive. In normal CUDA kernels, these accesses are usually cached and predictable.

Uniform Registers

Uniform registers store values that are common across a warp. They are useful when all lanes need the same value, such as a kernel parameter or block dimension.

The benefit is mainly power and resource efficiency rather than always direct performance. A uniform instruction may still consume an issue slot, but it avoids unnecessary per-lane register-file activity.

A simple conceptual distinction is:

$$\text{general register} = \text{one value per thread},$$

while

$$\text{uniform register} = \text{one value shared by the warp}.$$

The compiler decides when to use uniform registers. CUDA and PTX generally do not give the programmer complete direct control.

Predicate Registers

Predicate registers hold boolean conditions. They are used for branches and predicated execution.

A branch may conceptually be generated from:

```
if (x < limit) {
    y = a + b;
} else {
    y = c + d;
}
```

The SASS-level structure often includes:

```
ISETP condition, predicate
@!predicate BRA else_label
then_path_instructions
BRA end_label
else_label:
else_path_instructions
end_label:
```

This is only schematic. Real SASS includes convergence and synchronization details.

Zero Register

NVIDIA SASS has a special zero register, often written as `RZ`. It behaves like a register that always reads as zero. It can be used as an unused operand or as a source for instructions that need a zero input.

For example, a three-input integer add instruction can implement a two-input add by using zero as the third input:

$$d = a + b + 0.$$

PTX to SASS Is Not One-to-One

Three-Input Integer Add

PTX may expose a two-input add:

```
add.u32 d, a, b;
```

But SASS may use a three-input instruction such as `IADD3`:

```
IADD3 Rdst, Ra, Rb, RZ;
```

If the compiler finds another addend, it may fuse multiple PTX-level additions into one SASS instruction:

$$d = a + b + c.$$

The source-level code may appear to contain two additions, but the final SASS may contain one three-input add.

Logical Operation Fusion

Bitwise operations such as AND, OR, XOR, and NOT may be combined into `LOP3`. This instruction can implement arbitrary three-input boolean functions.

A three-input boolean function has $2^3 = 8$ input combinations. The output truth table therefore has 8 bits. `LOP3` can encode the desired truth table as an immediate value.

For inputs a, b, and c, the output is:

$$d_i = f(a_i, b_i, c_i)$$

for each bit position i. This makes `LOP3` powerful for bit manipulation and compiler optimization.

Floating-Point Reassociation Is Restricted

Floating-point arithmetic is not exactly associative:

$$(a + b) + c \neq a + (b + c)$$

in general, because rounding occurs after each operation. Therefore, the compiler often cannot freely reorder floating-point expressions unless fast-math assumptions or explicit source changes permit it.

This matters in optimization. A source expression may produce extra instructions unless the programmer writes it in a form that allows FMA fusion or avoids unnecessary multiplies.

Hardware-Level Explanation

SM Partitions and Warp Schedulers

Modern NVIDIA SMs are partitioned. A useful mental model is that an SM contains multiple subpartitions, each with:

- a warp scheduler,
- execution pipelines,
- register-file partitions,
- load-store capability,
- and access to shared SM resources.

The lecture emphasizes that many limits are really per partition rather than per whole SM.

If an SM has four partitions and each partition can schedule one warp instruction per cycle, then the whole SM can schedule at most:

$$4 \text{ warp instructions per cycle.}$$

This scheduling limit can become a bottleneck for vector-heavy code.

One Instruction Per Warp Scheduler Per Cycle

A key bottleneck is:

$$\text{issue rate} \leq 1 \text{ warp instruction per scheduler per cycle.}$$

Even if multiple pipelines exist, a scheduler cannot necessarily issue arbitrary numbers of instructions in one cycle. Therefore, a kernel with many scalar

integer, branch, and floating-point instructions can become instruction-issue limited.

This is different from tensor core kernels, where a single instruction may represent a large amount of work. Tensor core instructions amortize the scheduler overhead over many operations.

Pipeline Classes

The lecture discusses several pipeline categories:

- FP32 pipelines for floating-point add, multiply, and FMA.
- Integer pipelines for integer arithmetic and some compare or min-max operations.
- Special function units for reciprocal, square root, exponential, trigonometric approximations, and related operations.
- Load-store pipelines for memory instructions.
- Tensor core pipelines for matrix multiply-accumulate instructions.
- Branch and control-flow resources.

The exact mapping of instructions to pipelines is architecture-specific and sometimes non-obvious. For example, an instruction with a name that looks like a half-precision FMA may sometimes be used by the compiler as a convenient way to move or zero data through an otherwise available pipeline.

Thread-Level and Instruction-Level Parallelism

Latency hiding requires enough independent work. The lecture connects this to Little's Law.

A useful form is:

$$L = \lambda W,$$

where:

- L is the number of operations in flight,
- λ is throughput,
- W is latency.

For GPUs, this becomes:

$$\text{needed in-flight work} = \text{target throughput} \cdot \text{latency}.$$

If a memory operation has latency W, then enough warps and enough independent memory operations must be in flight to cover that latency.

There are two major forms of parallelism:

$$\text{latency hiding} = \text{thread-level parallelism} + \text{instruction-level parallelism}.$$

Thread-level parallelism comes from many active warps. Instruction-level parallelism comes from independent instructions within a warp, such as multiple independent loads before dependent arithmetic.

Register Pressure and Occupancy

Registers are a limited resource. If each thread uses more registers, fewer warps or thread blocks can fit on an SM.

Let:

$$R_{\text{SM}}$$

be the total registers per SM partition or SM, depending on the allocation granularity. If each thread uses R_{thread} registers and a block has T_{block} threads, then registers per block are approximately:

$$R_{\text{block}} = R_{\text{thread}} \cdot T_{\text{block}}.$$

The maximum number of resident blocks due to registers is bounded by:

$$B_{\text{reg}} = \left\lfloor \frac{R_{\text{SM}}}{R_{\text{block}}} \right\rfloor.$$

Occupancy is constrained by the minimum over registers, shared memory, warp limits, block limits, and architectural scheduling limits:

$$B_{\text{resident}} = \min\left(B_{\text{reg}}, B_{\text{shared}}, B_{\text{warp}}, B_{\text{block}}\right).$$

The lecture notes that register allocation granularity can create thresholds. Going from one register count to another may suddenly reduce occupancy if it crosses an allocation boundary.

Register Spilling

If the compiler cannot fit live values into the allowed register budget, it spills values to local memory. In SASS, local memory accesses may appear as instructions like stack stores and stack loads.

A simplified pattern is:

```
STL [stack_slot], Rvalue
...
LDL Rvalue, [stack_slot]
```

This is usually a bad sign because local memory is backed by the memory hierarchy. It may hit L1 or L2, but it is still far slower than registers.

However, the trade-off is not always trivial. Sometimes slightly more spilling with much higher occupancy can outperform zero spilling with low occupancy. The correct answer depends on the kernel.

Launch Bounds and Compiler Register Strategy

CUDA launch bounds can tell the compiler how many threads per block and blocks per SM it should try to support. This affects register allocation.

A representative annotation is:

```
__launch_bounds__(256, 8)
__global__ void kernel(float* out, const float* in) {
    // kernel body
}
```

This tells the compiler to compile under assumptions compatible with 256 threads per block and up to 8 blocks per SM, if possible. To satisfy this, the compiler may reduce register usage. If it cannot reduce register usage by scheduling and liveness optimization, it may spill.

The optimization trade-off is:

$$\text{higher occupancy} \leftrightarrow \text{lower registers per thread} \leftrightarrow \text{greater spilling risk.}$$

Branching and SIMT Control Flow

Volta-Style Independent Thread Scheduling

The lecture emphasizes that Volta and later architectures changed SIMT control flow significantly. Instead of a purely lockstep warp model, Volta introduced more flexible independent thread scheduling.

However, warps still execute instructions under a SIMT model. Divergent branches are handled through masks, predicates, branch instructions, and reconvergence mechanisms.

Divergent Branch Structure

Consider:

```
if (threadIdx.x < 16) {
    a = a + 1.0f;
} else {
    a = a - 1.0f;
}
```

A conceptual SASS-like control flow is:

```
set predicate based on threadIdx.x < 16
initialize convergence barrier
branch inactive side to else path
execute then path for active lanes
branch to end
re-enable lanes for else path
execute else path
reconverge
```

This requires extra instructions beyond the arithmetic itself:

- predicate set instruction,
- branch instruction,
- convergence setup instruction,
- branch synchronization or reconvergence instruction,
- and possibly extra unconditional branches.

Therefore, branch divergence has two costs:

$$\text{divergence cost} = \text{extra control instructions} + \text{serialized execution of paths}.$$

Predication Versus Branching

For small conditional operations, the compiler may use predication rather than full branching. Predication means that an instruction is executed only for lanes where a predicate is true.

A schematic form is:

```
@P0 instruction_for_true_lanes
@!P0 instruction_for_false_lanes
```

Predication avoids some branch machinery but can waste issue slots if many predicated instructions are inactive. Full branching can avoid executing long inactive paths, but it adds branch and reconvergence overhead.

The choice depends on:

- branch body size,
- expected divergence,
- instruction mix,
- compiler heuristics,
- and architecture.

Dependency Barriers and Scheduling

Compiler-Encoded Dependencies

Modern NVIDIA SASS encodes dependency and scheduling information in the instruction stream. The hardware still tracks some state, but much scheduling responsibility is in the compiler.

There are two broad cases:

- fixed-latency arithmetic instructions,
- variable-latency instructions such as memory loads.

For fixed-latency instructions, the compiler can often insert a known stall or schedule independent instructions between producer and consumer.

For memory instructions, latency is variable. A global load may hit L1, hit L2, or go to HBM. Therefore, the instruction stream uses dependency barriers.

Read and Write Barriers

A memory load has at least two relevant dependency moments:

- the address registers are no longer needed by the load instruction,
- the loaded data is available for consumers.

The lecture describes barrier concepts similar to:

- **read barrier**: tracks when source registers can be reused,
- **write barrier**: tracks when destination data is ready.

A later instruction may wait on a barrier mask before executing.

Conceptually:

$$\text{consumer instruction can issue} \iff \text{producer dependency barrier is satisfied.}$$

This is why SASS inspection can reveal dependency chains that are not obvious from source code.

Mathematical Reasoning

Arithmetic Intensity

Arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

It determines whether a kernel is likely to be compute-bound or bandwidth-bound.

If the peak compute throughput is P_{peak} and memory bandwidth is B_{peak}, then the roofline performance bound is:

$$P_{\text{attainable}} \leq \min\left(P_{\text{peak}}, IB_{\text{peak}}\right).$$

If:

$$IB_{\text{peak}} < P_{\text{peak}},$$

the kernel is bandwidth-bound. If:

$$IB_{\text{peak}} \geq P_{\text{peak}},$$

the kernel may become compute-bound or limited by another bottleneck such as instruction issue, occupancy, or synchronization.

Instruction Issue Bound

If each SM has S warp schedulers and each scheduler can issue at most one warp instruction per cycle, then the peak warp-instruction issue rate per SM is:

$$R_{\text{issue}} = S \text{ warp instructions per cycle.}$$

For $S = 4$:

$$R_{\text{issue}} = 4 \text{ warp instructions per cycle per SM.}$$

If a kernel requires N_{inst} warp instructions per output element group, then an issue-bound lower bound on cycles is:

$$T_{\text{issue}} \geq \frac{N_{\text{inst}}}{R_{\text{issue}}}.$$

This is why instructions that do more work per issue slot, such as tensor core instructions or TMA instructions, are valuable.

Memory Transaction Efficiency

For a memory-bound kernel, useful bandwidth is:

$$B_{\text{useful}} = \frac{\text{useful bytes}}{\text{time}}.$$

Physical bandwidth consumed is:

$$B_{\text{physical}} = \frac{\text{actual bytes transferred}}{\text{time}}.$$

Efficiency is:

$$\eta_{\text{mem}} = \frac{B_{\text{useful}}}{B_{\text{physical}}}.$$

Coalescing improves this ratio. Poor access patterns reduce it by causing extra memory transactions.

Read-Write Turnaround Overhead

The lecture notes that real HBM bandwidth is not simply one ideal number. A read-only kernel may reach a different fraction of peak than a mixed read-write kernel.

A simplified model is:

$$T_{\text{memory}} = T_{\text{read}} + T_{\text{write}} + T_{\text{turnaround}}.$$

The term $T_{\text{turnaround}}$ represents overhead from switching between reads and writes or other low-level DRAM effects. Therefore, a one-to-one read-write kernel may not achieve the same fraction of peak bandwidth as a pure read or pure write kernel.

Occupancy and Register Thresholds

If a kernel uses R_{thread} registers per thread and the register allocator rounds to a granularity g, the effective register usage may be:

$$R_{\text{effective}} = g \left\lceil \frac{R_{\text{thread}}}{g} \right\rceil .$$

Then the occupancy bound becomes:

$$B_{\text{reg}} = \left\lfloor \frac{R_{\text{available}}}{R_{\text{effective}} T_{\text{block}}} \right\rfloor .$$

This explains why a small increase in reported registers can suddenly reduce occupancy.

Code Walkthrough

A Minimal CUDA Indexing Kernel

A representative CUDA kernel is:

```
extern "C" __global__
void add_one(float* out, const float* in, int n) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < n) {
        out[tid] = in[tid] + 1.0f;
    }
}
```

The indexing expression:

$$\texttt{tid} = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}$$

typically lowers to SASS that:

- reads `threadIdx.x` from a special register,
- reads `blockIdx.x` from a special register,
- obtains `blockDim.x`,
- performs integer multiply-add,
- compares against n,
- computes a 64-bit address,

- loads a float,
- adds one,
- stores the result.

A schematic SASS-like sequence is:

```
S2R       Rthread, SR_TID.X
S2R       Rblock,  SR_CTAID.X
LDC       Rdim,    c[block_dim]
IMAD      Rid,     Rblock, Rdim, Rthread
ISETP.GE  P0,      Rid, n
@P0       EXIT
IMAD.WIDE Raddr,   Rid, 4, base_pointer
LDG       Rval,    [Raddr]
FADD      Rout,    Rval, 1.0
STG       [Raddr_out], Rout
EXIT
```

This is schematic rather than exact. The real SASS depends on architecture, compiler version, optimization level, and pointer aliasing assumptions.

Inline PTX for Warp Reduction

The lecture discusses a warp-level reduction instruction available in PTX for integer reductions. This can be useful for tricks such as absolute maximum reduction for floating-point values.

For nonnegative floating-point numbers, IEEE-style bit patterns preserve ordering when interpreted as unsigned or signed integer values in a suitable range. For absolute values, one may first clear the sign bit.

A conceptual CUDA pattern is:

```
__device__ float warp_abs_max_approx(float x) {
    x = fabsf(x);

    unsigned int bits = __float_as_uint(x);

    unsigned int max_bits;
    asm volatile(
        "redux.sync.max.u32 %0, %1, 0xffffffff;"
        : "=r"(max_bits)
        : "r"(bits)
    );

    return __uint_as_float(max_bits);
}
```

The idea is:

$$x \geq 0 \Rightarrow \text{larger float} \leftrightarrow \text{larger integer bit pattern}.$$

This avoids a multi-step floating-point warp reduction using shuffles. The important caveat is that edge cases such as NaNs, signed zeros, and negative values need careful treatment.

Compiler Reordering Across Loops

Consider:

```
extern "C" __global__
void sum_many(float* out, const float* in) {
    float data[8];

    #pragma unroll
    for (int i = 0; i < 8; ++i) {
        data[i] = in[threadIdx.x + i * blockDim.x];
    }

    float acc = 0.0f;

    #pragma unroll
    for (int i = 0; i < 8; ++i) {
        acc += data[i];
    }

    out[threadIdx.x] = acc;
}
```

The compiler may interleave loads and adds rather than preserving the source loop structure. It does this to:

- reduce register pressure,
- hide load latency,
- balance pipeline utilization,
- and satisfy occupancy constraints.

The source says:

$$\text{load all} \rightarrow \text{sum all}.$$

The SASS may become:

$$\text{load some} \rightarrow \text{add some} \rightarrow \text{load more} \rightarrow \text{add more}.$$

This is why source-to-SASS line mapping can be misleading.

Preventing Compiler Motion

Sometimes one wants to prevent the compiler from moving instructions across a boundary while investigating SASS. Two representative tricks are:

```
asm volatile("" ::: "memory");
```

and a no-inline device function:

```
__device__ __noinline__
float compiler_boundary(float x) {
    return x;
}
```

Then:

```
float y = compiler_boundary(x);
```

These tricks can inhibit some optimizations, but they are not perfect. The compiler may still move control-flow tests or optimize around them when it can prove correctness.

Restrict Qualifiers and Aliasing

Pointer aliasing can prevent optimization. If the compiler cannot prove that two pointers refer to distinct memory, it must be conservative.

A better signature is often:

```
extern "C" __global__
void kernel(float* __restrict__ out,
            const float* __restrict__ in,
            int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = in[i] * 2.0f;
    }
}
```

The qualifier `__restrict__` tells the compiler that the pointed-to regions do not alias. This can allow more aggressive load-store scheduling and caching assumptions.

Case Study: GELU and Approximate Tanh

GELU Formula

The Gaussian Error Linear Unit can be approximated as:

$$\mathrm{GELU}(x) = \frac{1}{2}x\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right).$$

A common constant is:

$$\sqrt{\frac{2}{\pi}} \approx 0.79788456.$$

Why Standard `tanh` Can Be Slow

The lecture highlights a performance issue in GELU kernels. A high-level call to `tanh` may compile to a sequence using special functions such as exponential and reciprocal, plus extra correction logic to satisfy strict precision requirements.

The exact implementation may involve:

- special function unit instructions,
- reciprocal or exponential approximations,
- branches or loops for accuracy correction,
- and extra arithmetic around edge cases.

This can create two problems:

1. It consumes special function pipeline resources.
2. It creates dependency chains that reduce instruction-level parallelism.

Approximate Tanh Through PTX

PTX exposes approximate math operations that may not be directly available as CUDA C++ intrinsics. Using inline PTX can request a lower-precision approximate tanh operation.

A schematic pattern is:

```
__device__ float tanh_approx(float x) {
    float y;
    asm volatile(
        "tanh.approx.f32 %0, %1;"
        : "=f"(y)
        : "f"(x)
```

```
    );
    return y;
}
```

The benefit is that the approximate instruction may lower to a much simpler SASS sequence. The trade-off is reduced accuracy.

The engineering question is:

Is the approximation error acceptable for the model?

For many neural network activation kernels, approximate math may be acceptable, especially when the surrounding computation already uses FP16, BF16, FP8, or approximate tensor core arithmetic.

Rewriting GELU to Save Instructions

A naive expression may produce separate multiply and add instructions. By rewriting the expression, one can encourage FMA formation.

Original conceptual form:

$$y = \frac{1}{2} x \left(1 + t\right),$$

where:

$$t = \tanh(z).$$

This can become:

$$y = \frac{1}{2} x + \frac{1}{2} x t.$$

A source rewrite may expose an FMA:

$$y = \mathrm{fma}\left(\frac{1}{2} x, t, \frac{1}{2} x\right).$$

The operation count becomes:

$$\text{one multiply} + \text{one FMA}$$

instead of:

$$\text{multiply} + \text{add} + \text{multiply}.$$

The reason the compiler may not do this automatically is floating-point reassociation and precision constraints.

Case Study: L2-Side-Aware Memory Copy

Split L2 Cache Model

The lecture discusses a low-level experiment involving NVIDIA L2 cache topology. Some GPUs have an L2 cache organized into multiple sides or slices. Address mapping determines which side an address belongs to.

A simplified model is:

$$\text{address} \rightarrow \text{L2 side} \rightarrow \text{L2 slice} \rightarrow \text{HBM partition}.$$

If an SM accesses data physically closer to one L2 side, latency and power may differ from accessing the other side.

Address Interleaving

The lecture mentions address interleaving at a granularity such as 4 KiB or 8 KiB, depending on architecture and measurement. A simplified mapping is:

$$\text{L2 side} = \left(\left\lfloor \frac{\text{address}}{4096} \right\rfloor \bmod 2 \right).$$

This is only a conceptual model. Actual mappings can depend on architecture and reverse-engineering results.

Persistent Threads and Side-Aware Copy

A side-aware memory copy assigns work so that an SM preferentially handles addresses mapped to the nearby L2 side.

The conceptual algorithm is:

1. Launch persistent thread blocks.
2. Determine which SM or SM group a block runs on.
3. Determine which address regions map to which L2 side.
4. Assign each block to copy regions local to its L2 side.
5. Avoid unnecessary cross-L2 traffic.

A schematic kernel structure is:

```
extern "C" __global__
void l2_side_aware_copy(float* dst,
                        const float* src,
                        size_t n) {
    int tid = threadIdx.x;
    int block = blockIdx.x;

    // Pseudocode only.
    // Real implementation needs SM identification,
    // address mapping, persistent scheduling,
    // and careful bounds handling.

    for (size_t tile = block; tile < n; tile += gridDim.x) {
        size_t i = tile * blockDim.x + tid;
        if (i < n) {
            dst[i] = src[i];
        }
    }
}
```

The performance gain may be modest in raw throughput, such as a few percent, but power savings can be larger if cross-chip or cross-L2 traffic is reduced.

Power Measurement

Nsight Compute is not ideal for direct power measurement because profiling changes execution behavior and may control clocks. A practical method is to use `nvidia-smi` during a steady-state loop.

A representative command is:

```
nvidia-smi --query-gpu=power.draw,clocks.sm,clocks.mem \
           --format=csv -lms 100
```

The important caution is that profiler clock control can make measurements nonrepresentative. Nsight Compute may run kernels under base clocks unless configured otherwise.

Performance Intuition

When SASS Inspection Helps Most

SASS inspection is most useful when:

- profiler counters show unexpected stalls,
- PTX looks reasonable but performance is poor,
- register pressure is higher than expected,

- a source expression generates too many instructions,
- a math function lowers to a slow sequence,
- compiler line mapping is confusing,
- occupancy changes unexpectedly,
- or the kernel is near peak and small bottlenecks matter.

When SASS Inspection Is Overkill

It is less useful when:

- the algorithm is obviously inefficient,
- memory access is obviously uncoalesced,
- PyTorch-level fusion or layout changes would dominate,
- the kernel is not on the critical path,
- or the implementation is still changing rapidly.

The optimization order should usually be:

algorithm → data layout → fusion → tiling → memory hierarchy → compiler output → SASS-level details.

Memory-Bound Versus Compute-Bound Kernels

A memory-bound kernel is limited by data movement:

$$T \approx \frac{\text{bytes moved}}{\text{bandwidth}}.$$

A compute-bound kernel is limited by arithmetic throughput:

$$T \approx \frac{\text{operations}}{\text{compute throughput}}.$$

But many real kernels are limited by other factors:

- instruction issue rate,
- special function throughput,
- branch divergence,
- register spilling,
- low occupancy,
- insufficient instruction-level parallelism,

- synchronization overhead,
- cache latency,
- or memory partition imbalance.

Instruction-Level Parallelism in Load-Heavy Code

For a sequence of independent loads:

$$r_0 = x[i_0], \quad r_1 = x[i_1], \quad \ldots, \quad r_k = x[i_k],$$

the compiler may issue many loads before consuming the results. This increases in-flight memory operations.

Then arithmetic can be interleaved:

$$\text{load} \rightarrow \text{load} \rightarrow \text{add} \rightarrow \text{load} \rightarrow \text{add}.$$

This balances register pressure against latency hiding.

Special Function Bottlenecks

Special functions such as exp, tanh, reciprocal square root, and sine may use special function units. If a kernel calls these functions heavily, it may become special-function-bound rather than FP32-bound or memory-bound.

A useful approximation is:

$$T_{\text{kernel}} \geq \frac{N_{\text{SFU}}}{R_{\text{SFU}}},$$

where:

- N_{SFU} is the number of special function instructions,
- R_{SFU} is the special function issue or throughput rate.

Approximate math can reduce N_{SFU} or replace slow sequences with faster ones.

Common Misconceptions

Misconception: PTX Is the Assembly That Runs

PTX is not usually the final assembly. It is an intermediate virtual ISA. SASS is what executes.

Misconception: Source Lines Map Cleanly to SASS

The compiler can fuse, move, delete, or split operations. A SASS instruction may correspond to multiple source lines, and one source line may produce many SASS instructions.

Misconception: Higher Occupancy Always Means Faster

Higher occupancy helps hide latency, but it can require lower register usage. If lower register usage causes spilling or worse instruction scheduling, performance can decline.

Misconception: The Compiler Will Always Find the Best Floating-Point Form

The compiler is constrained by floating-point correctness. It may not reassociate operations unless allowed. Manual expression rewrites can expose FMA opportunities.

Misconception: All Barriers Are the Same

The word barrier can refer to multiple unrelated mechanisms:

- thread-block synchronization,
- memory fences,
- dependency barriers in SASS scheduling,
- TMA barriers,
- asynchronous copy barriers,
- and branch reconvergence mechanisms.

They are not interchangeable.

Misconception: Register Bank Conflicts Are Always Worth Manually Optimizing

Modern NVIDIA compilers and hardware handle many register bank issues well. Unless writing SASS directly or diagnosing an extreme bottleneck, register bank conflicts are usually not the first optimization target.

Misconception: Profiler Speed-of-Light Percentages Are Absolute Limits

A profiler's speed-of-light model is an approximation. Real hardware has lower-level effects such as DRAM turnaround overhead, cache topology, bank conflicts, instruction-cache behavior, and power management.

Practical Takeaways

- Inspect SASS to understand what the GPU really executes.
- Use PTX for intent, but use SASS for performance diagnosis.
- Use Nsight Compute for runtime evidence, not just static disassembly.
- Watch for `STL` and `LDL`; they often indicate spilling.
- Be careful with `__launch_bounds__`; it can improve occupancy but cause spilling.
- Use `__restrict__` when pointer aliasing would otherwise block optimization.
- Check math functions such as tanh, exp, and reciprocal; they may lower to slow sequences.
- Rewrite floating-point expressions when it safely exposes FMA or removes extra instructions.
- Think in terms of instruction issue, not only FLOPs and bandwidth.
- Remember that modern GPU optimization requires balancing occupancy, instruction-level parallelism, memory locality, and compiler behavior.

Project or Experiment Ideas

Experiment 1: PTX Versus SASS Mapping

Write a CUDA kernel with simple integer additions:

```
out[i] = a[i] + b[i] + c[i];
```

Inspect PTX and SASS. Check whether the backend emits a three-input integer add.

Experiment 2: Floating-Point Reassociation

Compare two GELU-like expressions:

```
float y1 = 0.5f * x * (1.0f + t);
float half_x = 0.5f * x;
float y2 = fmaf(half_x, t, half_x);
```

Inspect SASS and measure runtime. Check whether the second version reduces instruction count.

Experiment 3: Register Pressure and Launch Bounds

Create a kernel with many live temporary values. Compile with different launch bounds:

```
__launch_bounds__(256, 1)
__global__ void k1(float* out, const float* in) { }

__launch_bounds__(256, 8)
__global__ void k2(float* out, const float* in) { }
```

Measure:

- registers per thread,
- occupancy,
- spills,
- runtime,
- and SASS instruction count.

Experiment 4: Approximate Tanh in GELU

Implement two GELU kernels:

1. one using standard `tanhf`,
2. one using approximate inline PTX.

Measure:

- throughput,
- error relative to standard GELU,
- special function utilization,
- instruction count,
- and end-to-end model impact if possible.

Experiment 5: Nsight Compute Stall Analysis

Profile a memory-bound vector kernel and a compute-heavy activation kernel. In Nsight Compute, inspect:

- warp stall reasons,
- not-issued samples,
- memory workload analysis,
- source-to-SASS mapping,

- achieved occupancy,
- and instruction mix.

Then modify the source and observe how the stall distribution changes.

Experiment 6: Power Measurement

Run a kernel in a loop and sample power using `nvidia-smi`. Compare:

- standard memory copy,
- vectorized memory copy,
- read-only kernel,
- write-only kernel,
- one-to-one read-write kernel.

Observe that peak bandwidth and power behavior differ by access pattern.

Final Mental Model

SASS is the real contract between the compiler and the GPU. PTX tells you what the program meant, but SASS tells you what the hardware will try to execute. Nsight Compute tells you how that execution behaved.

The most productive workflow is not to hand-write everything in assembly. It is to write CUDA, Triton, or PyTorch code with enough hardware intuition that the compiler emits good SASS. When performance is surprising, inspect SASS, connect it to profiler counters, and modify the source to change register pressure, instruction mix, memory behavior, branching, or math lowering.

At the lowest level, GPU performance is a balance among:

algorithmic efficiency, memory traffic, instruction issue, latency hiding, register pressure, occupancy,

The lecture's deepest lesson is:

> You do not study SASS because you always want to write SASS. You study SASS because every high-performance GPU program eventually becomes SASS, and the only way to fully understand the bottleneck is to understand what the machine actually received.